



FIT2096 - Games Programming

WEEK 01 LAB

Simple Inventory System

Welcome to **Games Programming!** Throughout the next twelve weeks you will be learning a variety of programming techniques for developing games within a game engine. By the end of this unit you will be able to create basic games including full mechanic design and an introduction to the visual design aspects of a game.

The core concepts you will learn are applicable in most major game engines, as they are all similarly designed. For the labs and assignments we will be making use of Unreal Engine 5.

This Lab is Assessed

Each of the labs from weeks 1 - 9 are assessed as part of the Learning Portfolio. Please make sure to check Moodle and OnTrack for detailed information on each of the tasks and what needs to be submitted.

Learning outcomes this week:

- Learn the Unreal Engine 5 Interface
- Build C++ classes using inheritance, polymorphism and pointers
- Integrate new code within an existing project



Overview - Getting started with Unreal Engine 5

For this semester we will be learning game programming with **Unreal Engine 5**, one of the commonly used engines in the games industry, alongside **Unity** and **Godot**.

Before starting this lab make sure that you check the guide for installing the required software on Moodle. There a number of steps and different applications to install, so it is important to follow the guide carefully. **You do not need to do this for the lab machines as the required software is already installed.**

This week we will be introducing you to Unreal Engine and the C++ programming language as well. If you have experience with C++ already this will serve as a refresher, and if not we'll be explaining it all as we go through, so no stress!

Unreal Engine 5 has high system requirements so be careful to ensure your own personal computer / laptop are capable of running the engine. The engine requirements are listed below

Minimum System Requirements

Operating System: Windows 10 64-bit OR MacOS Sonoma

Processor: Quad-Core Intel or AMD Processor. 2.5Ghz or faster

RAM: 8GB+

Video Card / DirectX Version: DirectX 11 or 12 Compatible Graphics Card

Unreal Engine and other required software are installed on all Windows PCs within G11A, G11B and G12B 14 Rainforest Walk. You have access to these rooms when classes are not running.

Warning - Rider Link

When first opening Rider it may ask you if you want to install Rider Link in the project or the engine. Make sure you close this prompt without installing. It can break projects!

Congratulations on your new role as programmer in the company. We have been working on an RPG inventory management system for our upcoming game. The previous programmer had finished the UI code and had started to program the C++ code before they had to leave.

For your first task you will need to take this base code and finish creating the required C++ classes to create, view and delete items from the inventory. We are providing you with instructions on how to create the needed C++ classes.

Good luck on the task and let's go!

- Project Lead



Opening the Base Code

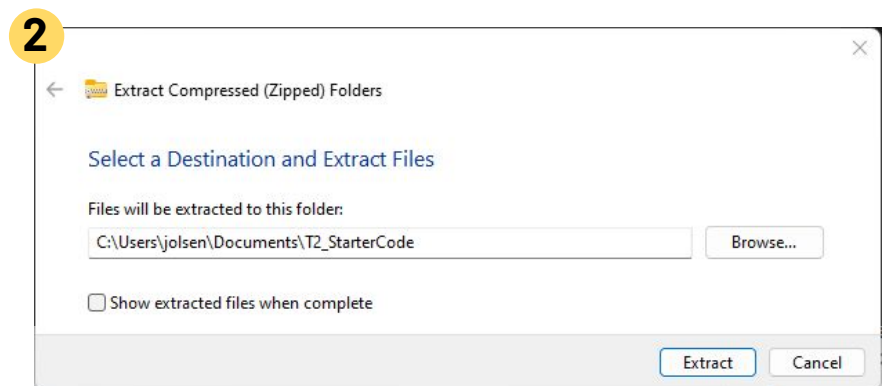
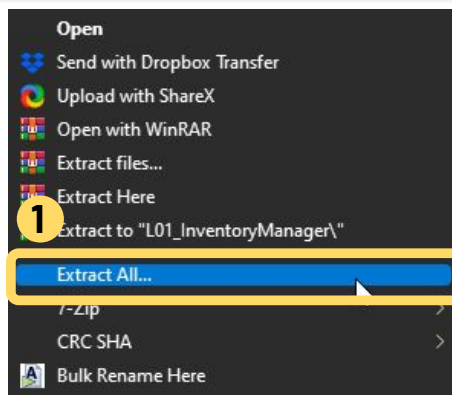
Download the base code from Moodle under the Week 1 section. The provided code has the full UI implementation for the inventory system as well as all required assets. There are also some pre-created C++ code files that you will need to modify.

1. Extract Project

To begin **extract** the project from the **provided zip file** to a suitable location. **On the lab machines you will need to extract this to the local documents folder.** All provided files for this unit will be within a standard zip file so you can use the default archive application or your preferred application for this process.

Warning - Allowed Locations

Unreal Engine cannot open projects stored on network drives. When using the lab machines always keep projects in the local Documents folder (C:\Users\AUTHCATE\Documents)

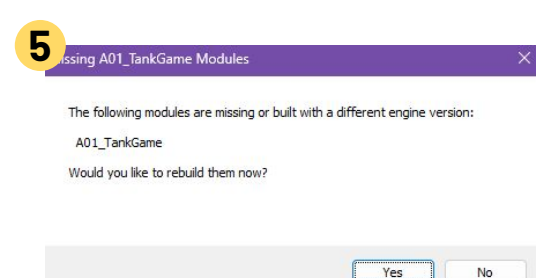
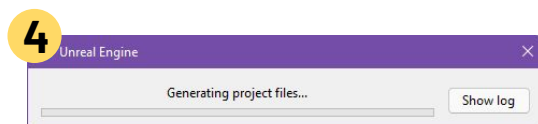
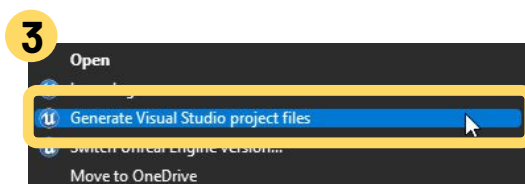


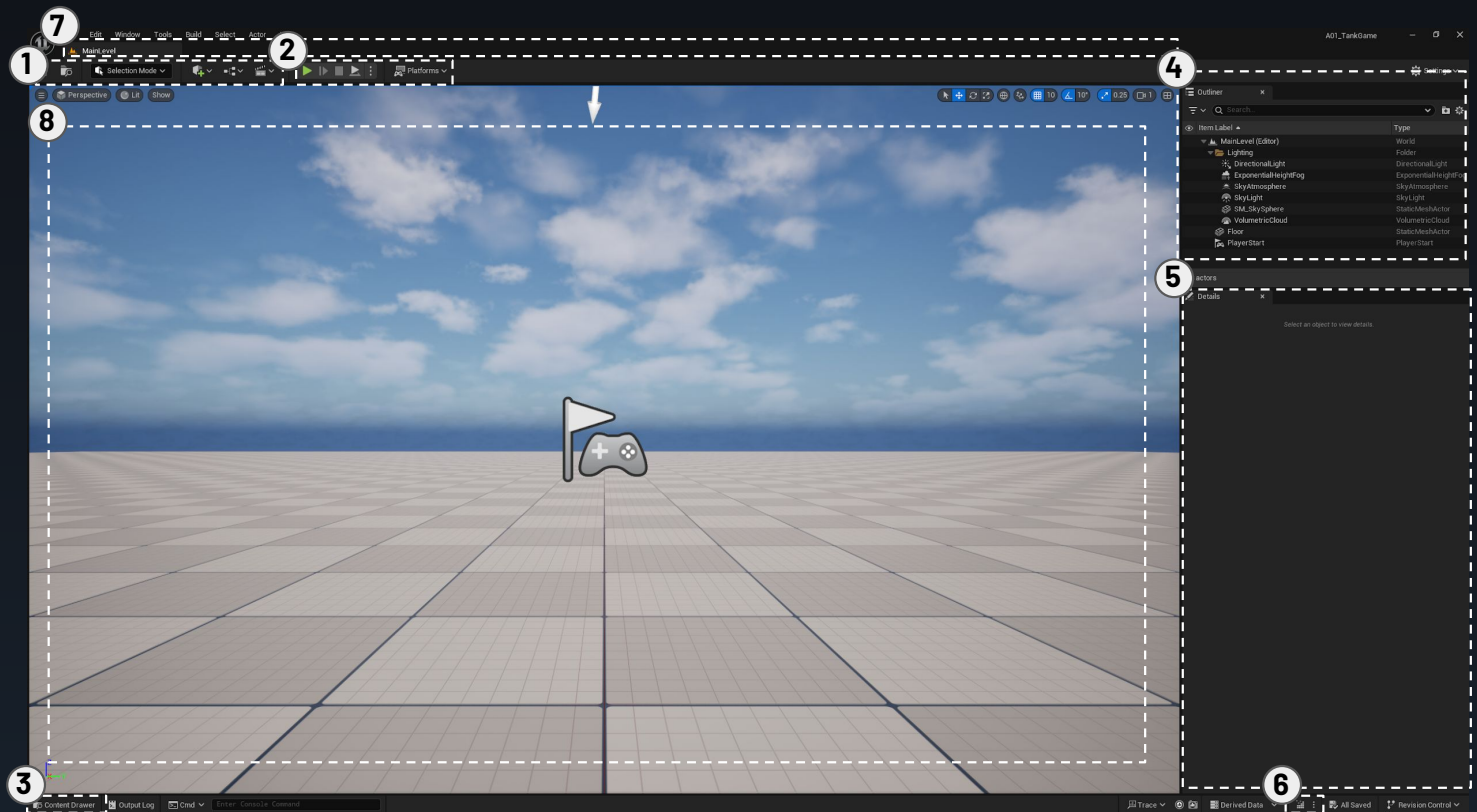
2. Generate Required Files

Unreal requires a number of generated files to work. These are unique to every PC so are generated on first launch. We do not include these in submissions. When you submit your assignments and commit to GitLab you will also be excluding these files. Instructions on how to do this can be found on Moodle and the Ed Forums.

Right click on the **.uproject file** in the project folder (*T2_InventorySystem.uproject*) and select the option **Generate Visual Studio project files**.

Next **open** the **.uproject file**. A new window will appear saying that parts of the project need to be rebuilt. Select **Yes** to start the process. Once finished Unreal Engine will open.





1. Common Tasks

The most common editor options are included here. Allows for quickly creating objects, accessing content browsers, level blueprints and cinematics

2. Play Options

Options for playing the open level. Default is open in selected viewport. Can open in new window or test on other platforms

3. Content Drawer

Opens a drawer with all content within the project. Separated into “Content” and “C++ Classes”.

CTRL + Space to open the drawer quickly.

Can be docked to be permanently open

4. World Outliner

Shows all objects that are present within your current level. Can be grouped and nested as required

5. Details Panel

Shows all details about a selected object. Includes transformation information, exposed variables and other settings.

6. Compile Options

Options for compiling C++ code. Clicking the button will compile code. Clicking the three dots will open a menu for more options to select from.

7. Tabs

Collection of tabs that have been opened. By default the selected level will be open. Can have open other object blueprints, level blueprints, cinematics, etc.

8. Viewport

The window to your game world. Shows the scene and objects within

Examining the Project

Let's take a look at the existing structure of the project and the current level of implementation left to us by the previous programmer.

3. Testing the Application

When you first open the application you will notice a blank arena. This is based on the third person template and something we will continue to use next week.

Click on the three dots next to the **Play button** and then select **New Editor Window (PIE)**. You will see that a UI appears... although you will notice that it does not do anything yet.

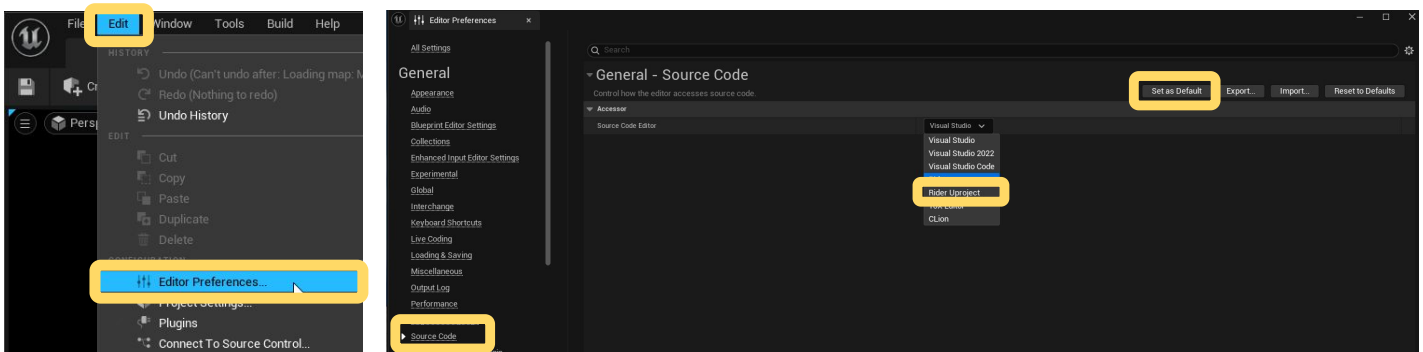


We will be adding functionality to this inventory screen as part of the lab tasks for this week.

4. Changing Default IDE

By default Unreal Engine opens projects with Visual Studio (Windows) or Xcode (MacOS). Neither of these are preferred when developing for Unreal and can make our lives harder. For this unit we ***strongly*** recommend using Rider. It is cross platform and has full integration with Unreal Engine.

To change the default IDE click on **Edit** in the top menu and go to **Editor Preferences**. Select **Source Code** from the menu on the left and then set **Source Code Editor** to be **Rider Uproject**. Click **Set as Default** to apply the settings across all projects. When prompted **restart** Unreal Engine.



Examining the Project

Let's take a look at the existing structure of the project and the current level of implementation left to us by the previous programmer.

5. Content Browser

Let's take a look at what content we have been provided with by the former programmer.

Open the **Content Browser** from the **bottom button** or by pressing **CTRL+Spacebar**.

The content browser is broadly divided into two sections; Content and C++ Classes. Blueprints, Textures, 3D Models and other assets are in the Content section.

Inside of the **C++ Classes** section you can see a folder named after the project. Within this folder there are two existing classes in the application.

AT2_InventorySystemCharacter:

This class is one that defines the player character and how they are able to interact with the Game World. This week we will be creating some new variables and functions inside this class to define our Inventory

AT2_InventorySystemGameMode:

This is our GameMode class. All projects have at least one GameMode. You can define rules for your games in here as well as setting up defaults.

InventoryWidget:

InventoryWidget defines the C++ code behind our User Interface. We will be updating functionality in this class to display our inventory items to screen. In later weeks we will examine creating our own custom user interfaces in more detail.



Examining the Project

6. Understanding the Problem

To complete the Inventory Manager you will need to create a new class called Item that will represent what an Item is within the game. For this week this will involve a Name and Description, however in future weeks we will extend this by allowing the player to use items for various effects.

We will also hard-code a number of specific item names for this week as well. These will match names that the provided UI classes expect to see. These are:

- Health
- Speed
- Freeze
- Shrink

In addition to creating an Item class, we will also need to modify the character class so that it knows about Items and stores a collection of them as an inventory. This will also include the ability to get and delete items.

Finally updates to the Inventory Widget are also needed to finish the expected behaviour of the game.

All of these changes will be done in C++.



Disabling Rider's Broken AI Autocomplete

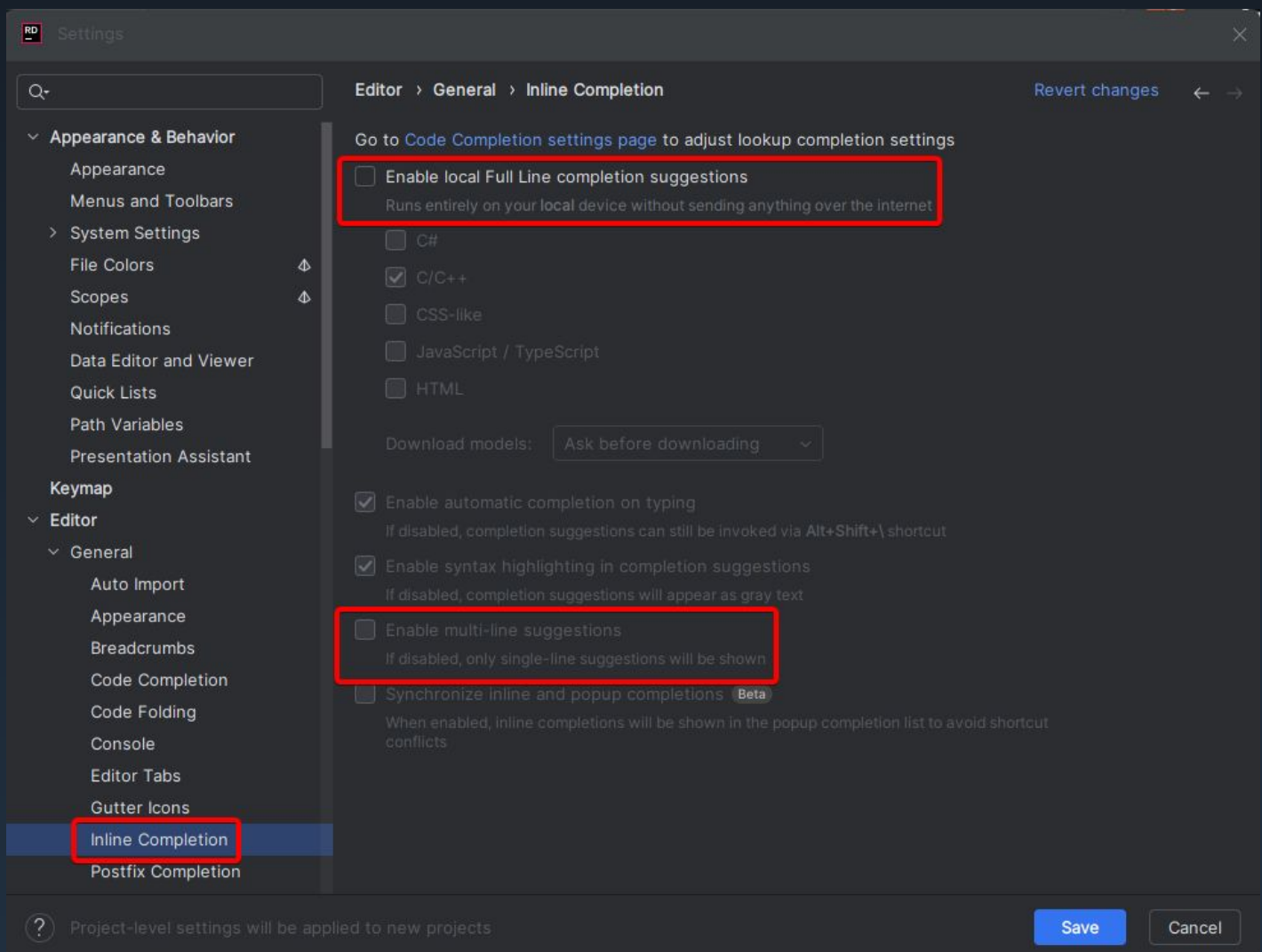
Before going any further into the programming of our lab tasks. We first will want to make a slight change to Rider's default settings.

In a relatively new update Rider has enabled local Full Line completion suggestions based on an AI solution run locally. Unfortunately this does not work well for Unreal Engine and mostly gives incorrect answers. Because of this we strongly recommend disabling the feature

With your code open in Rider complete the following

- Open the Settings screen (File -> Settings)
- Open up Editor -> General -> Inline Completion
- Ensure that "Enable multi-line suggestions" is unselected
- Ensure that "Enable local full line completion suggestions" is unselected

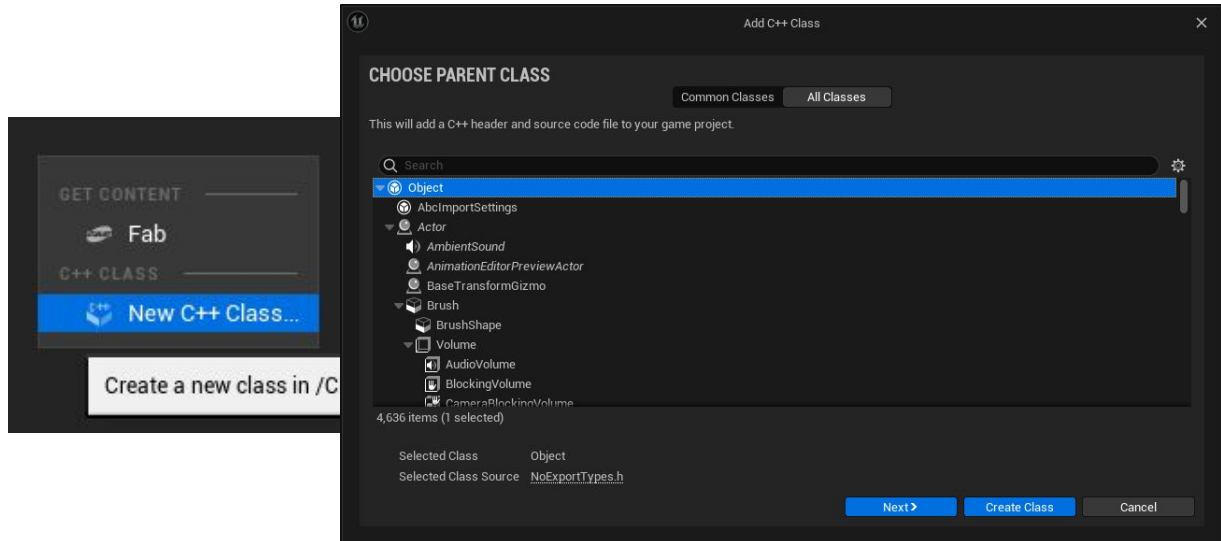
Click the Save button to finish up this step.



Creating the Item Class

7. Creating the Item Class

To create a new C++ class, open the **Content Browser** in Unreal, select the **C++ classes**, **right click** and select **New C++ Class**. This will bring up a window where you can select the parent class for your class. Click **All Classes** then select **Object**. Name the class **ItemObject** and click **Create Class**.



Header File:

Inside of the header file we need to define the variables and functions the Item class will have. In C++ we also need to specify the privacy into categories. We want our variables to be protected and all functions to be public.

Add the following inside of the class (under GENERATED_BODY()) to define the variables.

protected:

```
FString ItemName;
FString ItemDescription;
```

Unreal Engine does not use standard strings and instead have their own called FString. This offers additional options for formatting and inserting other data types into strings. You can find out more information using the Unreal Engine documentation.

<https://dev.epicgames.com/documentation/en-us/unreal-engine/fstring-in-unreal-engine>

Next we need to define Item's functions. We will need a single default constructor and functions for getting and setting each of the variables. Add the following code

public:

```
UItemObject();

void SetName(FString Name);
void SetDescription(FString Description);

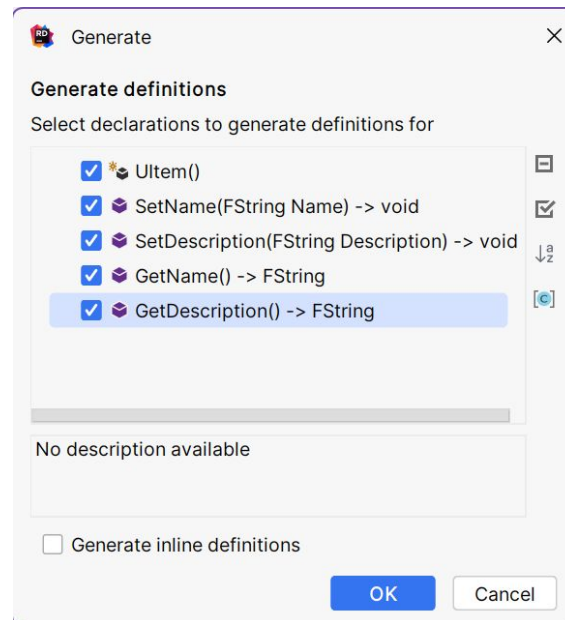
FString GetName();
FString GetDescription();
```

7. Creating the Item Class (contd.)

Header File:

With these definitions done, the header file is complete. Before we move over to the CPP file however, we can use Rider's inbuilt features to generate empty function stubs for us inside the CPP file.

Right click inside the header file and select **Generate Code**, select **Definitions** and **tick all functions** and **select Ok**. This will create all the definitions and open the CPP file automatically.



CPP File:

We'll first start with the constructor to set all the default values for our variables, so add the following to **UItemObject()**:

```
ItemName = FString("None");
ItemDescription = FString("A default description");
```

Set up the remaining functions as follows:

```
void UItemObject::SetName(FString Name) {
    ItemName = Name;
}

void UItemObject::SetDescription(FString Description) {
    ItemDescription = Description;
}

FString UItemObject::GetName() {
    return ItemName;
}

FString UItemObject::GetDescription() {
    return ItemDescription;
}
```

Remember because our variables are protected, we can't access them elsewhere. Creating these functions allows us to share them with other classes.

8. Modifying the character to manage items

To use the items we need to make changes to our character. Let's do that now

Header File:

In **Rider**, open **T2_InventorySystemPlayerCharacter.h** and add the following lines to a **protected:** section:

```
void CreateDefaultItems();  
UPROPERTY()  
    TArray<UItemObject*> InventoryItems;
```

The function `CreateDefaultItem` will be responsible for adding all our starting items to the inventory. The `TArray` is our actual Inventory.

In Unreal, we use **TArray** for our dynamic arrays as they are managed by Unreal, memory efficient, and convenient! We're storing a `TArray` of **UItemObject pointers (*)** as we want to be able to store the exact items the player has used and picked up, not copies of them.

Because this `TArray` is a pointer, we need to store it as a `UPROPERTY` to ensure Unreal doesn't garbage collect it randomly on us!

Make sure to add an `#include` to the top of the file for our new `UItemObject` reference:

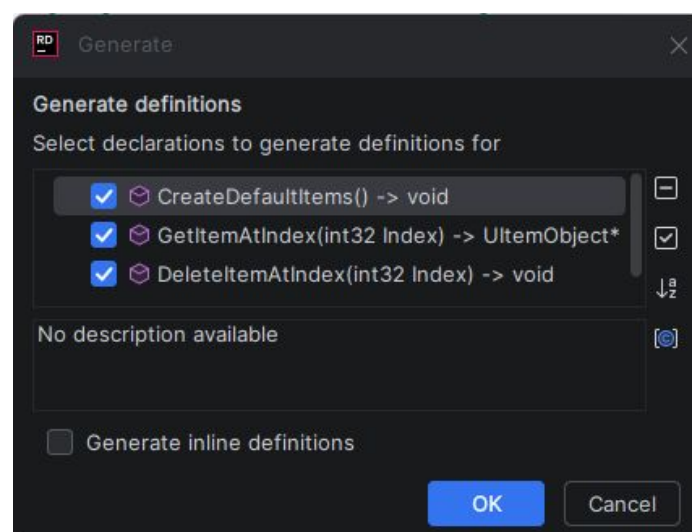
```
#include "ItemObject.h"
```

Add the following two functions to a **public** section:

```
UItemObject* GetItemAtIndex(int32 Index);  
void DeleteItemAtIndex(int32 Index);
```

These functions will allow us to provide indices and either get the item in our inventory at that index, or delete it! By putting them in the public section, it allows us to call these functions from other classes, like our Inventory Widget!

Remember to once again **right click** inside of the **header** file and click **Generate Code** to generate our function's definitions!



8. Modifying the character to manage items (contd.)

Next open **T2_InventorySystemPlayerCharacter.cpp** and make the following changes

Add the following to **GetItemAtIndex**:

```
if(Index ≥ InventoryItems.Num() || Index < 0) {
    return nullptr;
}

return InventoryItems[Index];
```

Here we check to see if our index is valid, by making sure it is somewhere between 0 and the number of items in the array. If it is not valid we return a null pointer, otherwise we return the item.

Next, **add** the following code inside of **DeleteItemAtIndex**:

```
if(Index ≥ InventoryItems.Num() || Index < 0) {
    return;
}

InventoryItems.RemoveAt(Index);
InventoryWidget→RefreshInventory(InventoryItems);
```

Here again we check to see if our index is valid, if not, we end the function here by using the **return** statement. If we do have a valid index we use the **RemoveAt** function from our TArray to remove the item at that location. Lastly, we call the **RefreshInventory** function from the **InventoryWidget** class to update the UI. This function does not exist yet so there will be an error

Finally **add** the following code into the **CreateDefaultItems** function:

```
UItemObject* TempItem;

TempItem = NewObject<UItemObject>();
TempItem→SetName("Health");
TempItem→SetDescription("A potion to recover lost health");
InventoryItems.Add(TempItem);

TempItem = NewObject<UItemObject>();
TempItem→SetName("Shrink");
TempItem→SetDescription("Makes you shrink for 10 seconds");
InventoryItems.Add(TempItem);

TempItem = NewObject<UItemObject>();
TempItem→SetName("Freeze");
TempItem→SetDescription("Makes you freeze time for 10 seconds");
InventoryItems.Add(TempItem);

InventoryWidget→RefreshInventory(InventoryItems);
```


8. Modifying the character to manage items (contd.)

Add the following inside of **BeginPlay** at the end, but still **inside**, the **if(APlayerController* ...)** statement:

```
CreateDefaultItems();
```

Now when the game starts, we generate a number of basic items to display in our menu.

What just happened?

The first thing we do is declare a variable of type **UItemObject*** called TempItem which we can use to set up our item objects. At this point it is an empty pointer with no item

Next, we create a new object using the **NewObject** function, which creates a new default object for us.

We then set it's name and description using the two functions we made earlier, and then lastly we add it to our **InventoryItems** array using the **Add** function from **TArray** which adds our TempItem to the **InventoryItems** array.

9. Refreshing the Inventory

To begin this step, open **InventoryWidget.h** and **add** a **new function** in the public section:

```
void RefreshInventory(TArray<UItemObject*> Items);
```

This function will be used to take our items and display them to screen. Right click on the file and generate a definition for this function.

Make sure to add the following include statement above the other include statements.

```
#include "ItemObject.h"
```

Open **InventoryWidget.cpp** and **add** the **following code**:

```
for(UItemButtonWidget* Btn : ButtonList)
{
    Btn->UpdateItemIcon(*ImageMap.Find("None"));
}

for(int i = 0; i < Items.Num(); i++)
{
    if(i > ButtonList.Num() - 1)
    {
        return;
    }
    ButtonList[i]->UpdateItemIcon(*ImageMap.Find(Items[i]->GetName()));
}
```

This goes through each of buttons in this user interface, and then sets them to a default image. After that, we loop through all of the items we provided and set the button images using the Item name. We also do a quick check to see if we've provided too many items for our buttons to show.

9. Refreshing the Inventory

Add the following to the end of **OnButtonWasClicked**:

```
UItemObject* SelectedItem = Owner→GetItemAtIndex(SelectedItemIndex);

if(SelectedItem) {
    ItemNameTextBlock→
        SetText(FText::FromString(SelectedItem→GetName()));
    ItemDescriptionTextBlock→
        SetText(FText::FromString(SelectedItem→GetDescription()));
}
```

First, we get the index of the selected item by finding the position of the button in our button list using the **Find** function from **TArray** which takes in an object and returns the position of it in the array. Next, we get the **UItemObject** inside of our Character's inventory by using the **GetItemAtIndex** function we made earlier.

Then, we check to see if that selection was valid, if so, we set the text block for the name as well as the description to that of our item's name and description.

Lastly, add the following to **OnDeleteButtonClicked**:

```
Owner→DeleteItemAtIndex(SelectedItemIndex);
ItemNameTextBlock→SetText(FText::FromString("No Item Selected"));
ItemDescriptionTextBlock→
    SetText(FText::FromString("No Item Selected"));
```

This calls the **DeleteItemAtIndex** function we made earlier from our character class, then sets the text for both the item name and item description text box back to No Item Selected.

And that's it! That's all of the code for today, so make sure you **save** your work in **Rider** before returning to **Unreal**.

10. Compiling the Code

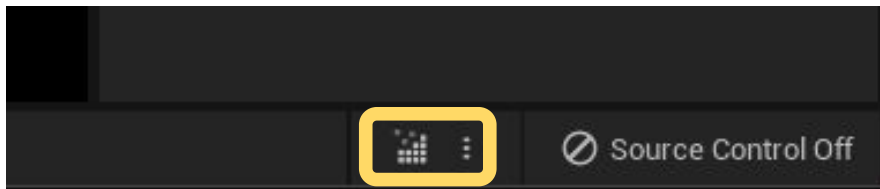
With the changes to code finished we need to compile the code so that we can test the application. Compiling code in Unreal Engine is a bit different than what you are used to in other programming units.

Instead of compiling within the Rider/Visual Studio/Xcode we instead must compile the code from within the Unreal Engine IDE. This is because the IDE is in-itself an instance of the engine running and compiling within Rider would cause a new instance of Unreal Engine to open.

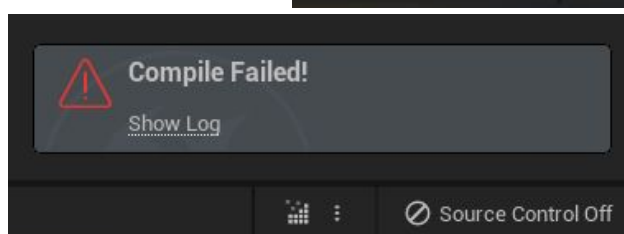
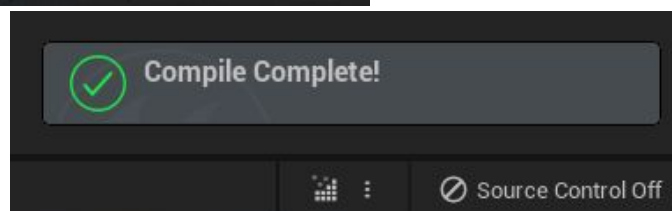
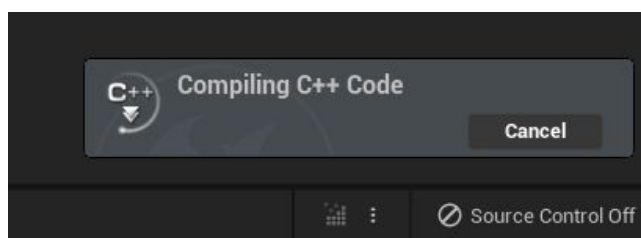
Debugging Bonus

Whilst this behaviour of opening a new instance of the engine when compiling from Rider may sound weird it is extremely useful when debugging as it allows us to use standard debugging tools.

To compile your project **click** the small button in the **bottom right corner** of Unreal Engine that looks like a **chart of dots** (image on right). Depending on the system's power; this could take a couple minutes.



If there are any compiler issues then it will display a message along with a link to view the error log. In severe cases where there is a more severe compiler issue it is possible that the engine will crash and require a manual recompile from Rider. We will discuss resolving these issues in more details in upcoming weeks, for now if you encounter these issues let your demonstrator know so they can assist in resolving it.

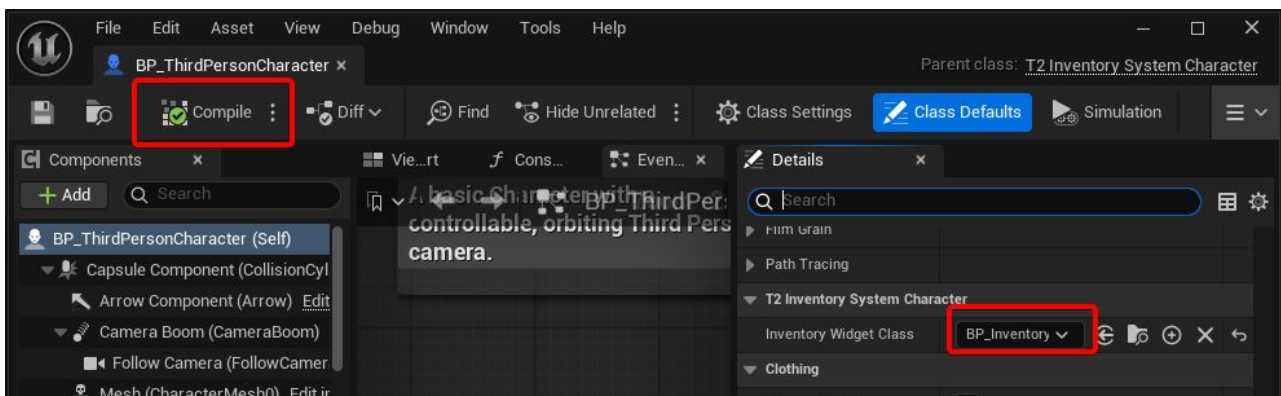


11. Updating the Character Blueprint

With the compilation done, there is one last step we need to complete. We need to give the Player Character a specific UI object to display to screen.

Open the Content Browser from the bottom of Unreal Engine and navigate to Content > Third Person > Blueprints and open BP_ThirdPersonCharacter. This is the Blueprint editor which we will use for setting variables that are more challenging in C++.

In the details panel on the right side, look for Inventory and set its value to be BP_InventoryWidget. Press the compile button in the top left, save and return to the main editor.



12. Testing the Application Again

Once compilation is finished it is time to test the application again. **Press the Run button** in the **Unreal Editor** to view the completed inventory manager. Assuming everything is working correctly you should see a number of items with their corresponding images on the inventory screen.

Selecting an item should cause its details to appear on the right side of the screen under the details section. Pressing the delete button should delete the item from the inventory and reshuffle the remaining items.

With this the application and testing is done and complete for this week. The next step will be to commit your project to a new git repository. This can be done by following the instructions in the [Connecting Unreal Projects to Git](#) document in week 1

Warning - Local Drives Wiped

Please note that the **documents folder on the lab computers is wiped at the end of each day**. This means if your work is not uploaded to git (or saved elsewhere at the end of the class) you may lose your work. Since we're using git this won't be a problem, but it's important to remember to push your work at the end of each lab.